

PRAKTIKUM SISTEM OPERASI

MODUL

(Sekuensial dan Konkuren)



Oleh:

Farhan Kurniawan (2311104073)

PROGRAM STUDI REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

JURNAL/TP

Soal

1. [10 Poin] Selain hardware (memory), batasan maksimal proses dapat ditentukan dengan secara software. Pada Linux maksimal proses adalah 4194303 proses (64 bit) dan 32767 proses (32 bit) dapat dilihat melalui perintah `$cat /proc/sys/kernel/pid_max`

Carilah pada source code Xinu yang memberi batasan mengenai banyaknya proses yang bisa dibuat! Berapa maksimal proses dalam Xinu? Ubah menjadi maksimal 150 proses!

2. [20 Poin] Jalankan kode `main_ex1`!
3. [20 Poin] Jalankan kode `main_ex2`!
4. [20 Poin] Jalankan kode `main_ex3`!
5. [30 Poin] Buatlah 2 proses produser dan konsumen. Produser memproduksi angka integer dari 1-1000. Konsumer mengkonsumsi integer yang diproduksi oleh produser dan menampilkannya! (Gunakan variabel global bertipe `int32` bernama `n` yang digunakan secara bersama oleh kedua proses)

```
int32 n = 0; //global variabel

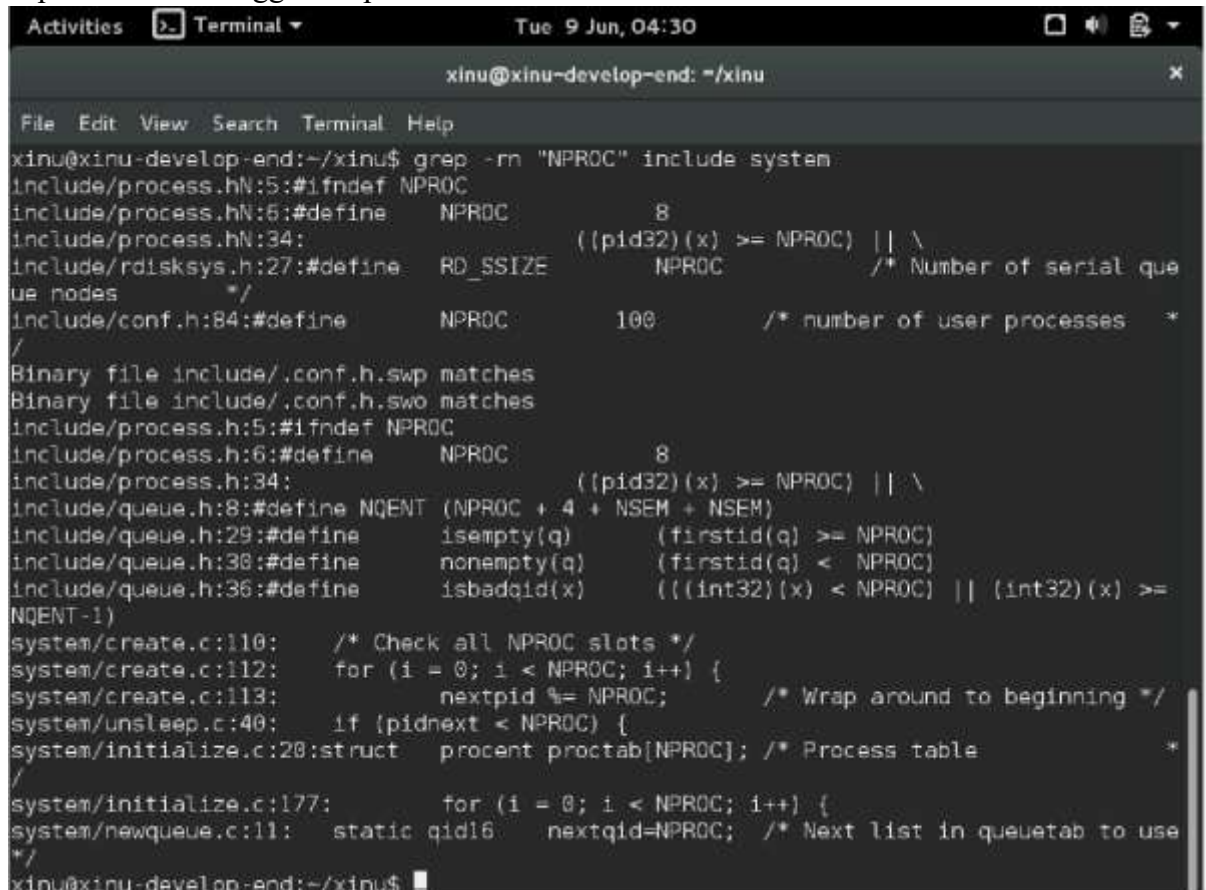
void produser(void){
    int32 i;
    for (i=1; i<=1000; i++){
        n++;
    }
}

void konsumer(void){
    int32 i;
    for (i=1; i<=1000; i++){
        printf("Nilai dari n adalah %d\n",n);
    }
}
```

Hasil dari program ini cukup mengejutkan (tidak akan sesuai dengan intuisi awal). Jelaskan mengapa hasilnya seperti itu!

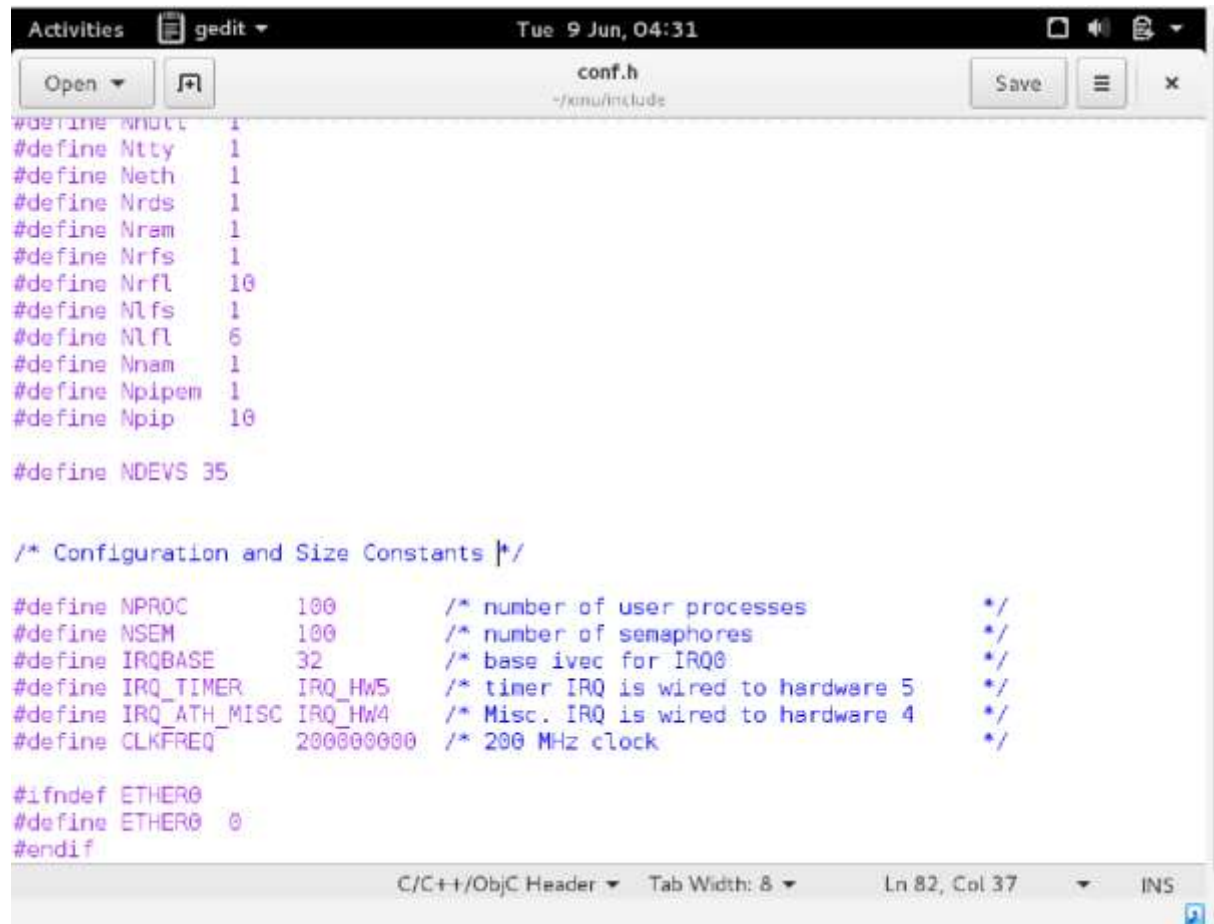
Jawaban

1. Batas maksimum proses pada Xinu terdapat pada file include/conf.h dengan konstanta NPROC. Nilai awal NPROC adalah 100 yang menunjukkan jumlah maksimum user process yang dapat dibuat. Nilai tersebut kemudian diubah menjadi 150 sehingga sistem dapat membuat hingga 150 proses.



```
xinu@xinu-develop-end: ~/xinu
File Edit View Search Terminal Help
xinu@xinu-develop-end:~/xinu$ grep -rn "NPROC" include system
include/process.hN:5:#ifndef NPROC
include/process.hN:6:#define      NPROC          8
include/process.hN:34:          ((pid32)(x) >= NPROC) || \
include/rdisksys.h:27:#define    RD_SSIZE      NPROC      /* Number of serial que
ue nodes */
include/conf.h:84:#define      NPROC          100      /* number of user processes */
/
Binary file include/.conf.h.swp matches
Binary file include/.conf.h.swo matches
include/process.h:5:#ifndef NPROC
include/process.h:6:#define      NPROC          8
include/process.h:34:          ((pid32)(x) >= NPROC) || \
include/queue.h:8:#define NQENT (NPROC + 4 + NSEM + NSEM)
include/queue.h:29:#define      isempty(q)      (firstid(q) >= NPROC)
include/queue.h:30:#define      nonempty(q)      (firstid(q) < NPROC)
include/queue.h:36:#define      isbadqid(x)      (((int32)(x) < NPROC) || ((int32)(x) >=
NQENT-1))
system/create.c:110:          /* Check all NPROC slots */
system/create.c:112:          for (i = 0; i < NPROC; i++) {
system/create.c:113:              nextpid %= NPROC;          /* Wrap around to beginning */
system/unsleep.c:40:          if (pidnext < NPROC) {
system/initialize.c:20:struct    procent proctab[NPROC]; /* Process table */
/
system/initialize.c:177:          for (i = 0; i < NPROC; i++) {
system/newqueue.c:11:  static qid16  nextqid=NPROC; /* Next list in queuetab to use
*/
xinu@xinu-develop-end:~/xinu$
```

Sebelum Diedit:



```
Activities gedit Tue 9 Jun, 04:31
Open conf.h Save
~/xmu/include

#define NIOCT 1
#define Ntty 1
#define Neth 1
#define Nrds 1
#define Nram 1
#define Nrfs 1
#define Nrfl 10
#define Nlfs 1
#define Nlfl 6
#define Nnam 1
#define Npipem 1
#define Npip 10

#define NDEVS 35

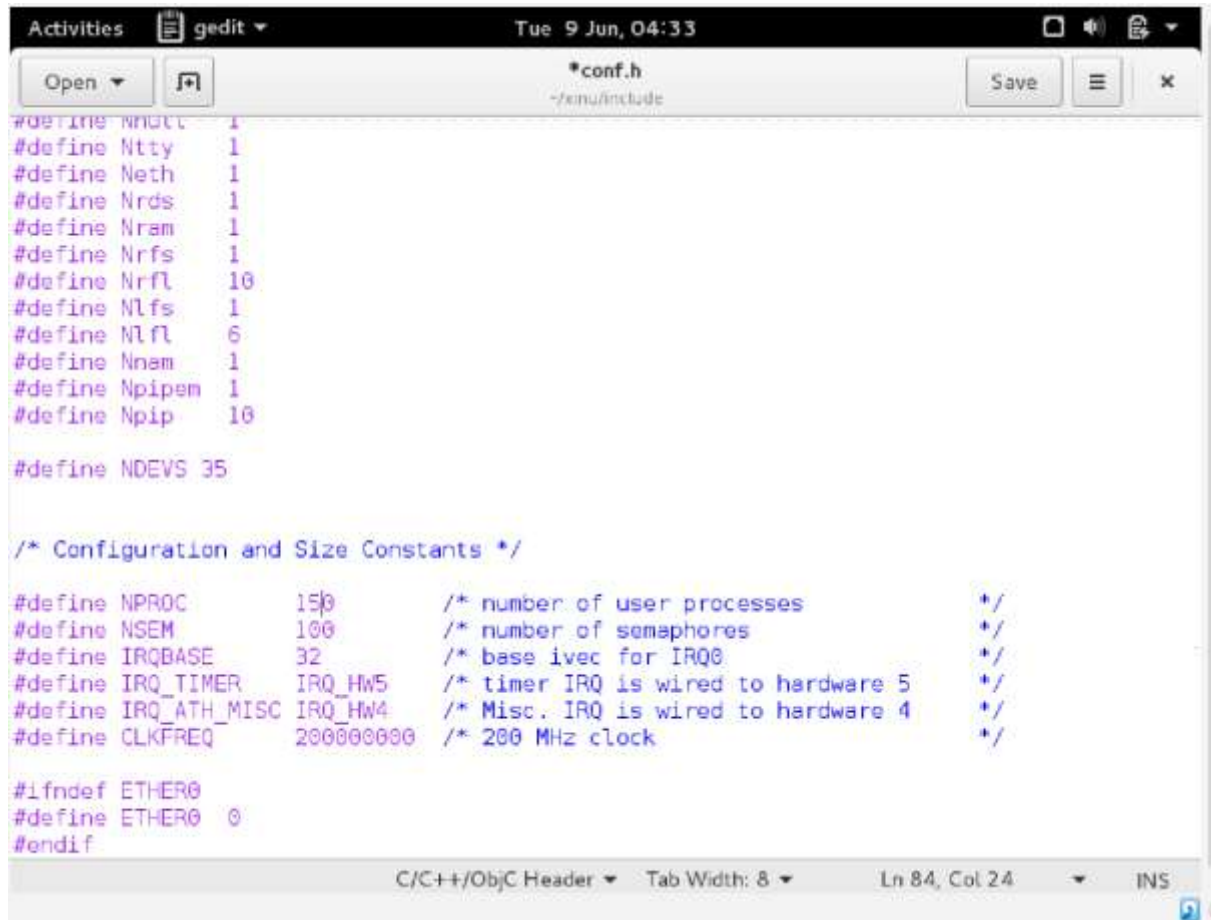
/* Configuration and Size Constants */

#define NPROC 100 /* number of user processes */
#define NSEM 100 /* number of semaphores */
#define IRQBASE 32 /* base ivec for IRQ0 */
#define IRQ_TIMER IRQ_HW5 /* timer IRQ is wired to hardware 5 */
#define IRQ_ATH_MISC IRQ_HW4 /* Misc. IRQ is wired to hardware 4 */
#define CLKFREQ 200000000 /* 200 MHz clock */

#ifndef ETHER0
#define ETHER0 0
#endif

C/C++/ObjC Header Tab Width: 8 Ln 82, Col 37 INS
```

Sesudah Diedit:



```
Activities gedit Tue 9 Jun, 04:33
*conf.h
~/emu/include

#define NMSG 1
#define Ntty 1
#define Neth 1
#define Nrds 1
#define Nram 1
#define Nrfs 1
#define Nrfl 10
#define Nlfs 1
#define Nlfl 6
#define Nnam 1
#define Npipem 1
#define Npip 10

#define NDEVS 35

/* Configuration and Size Constants */

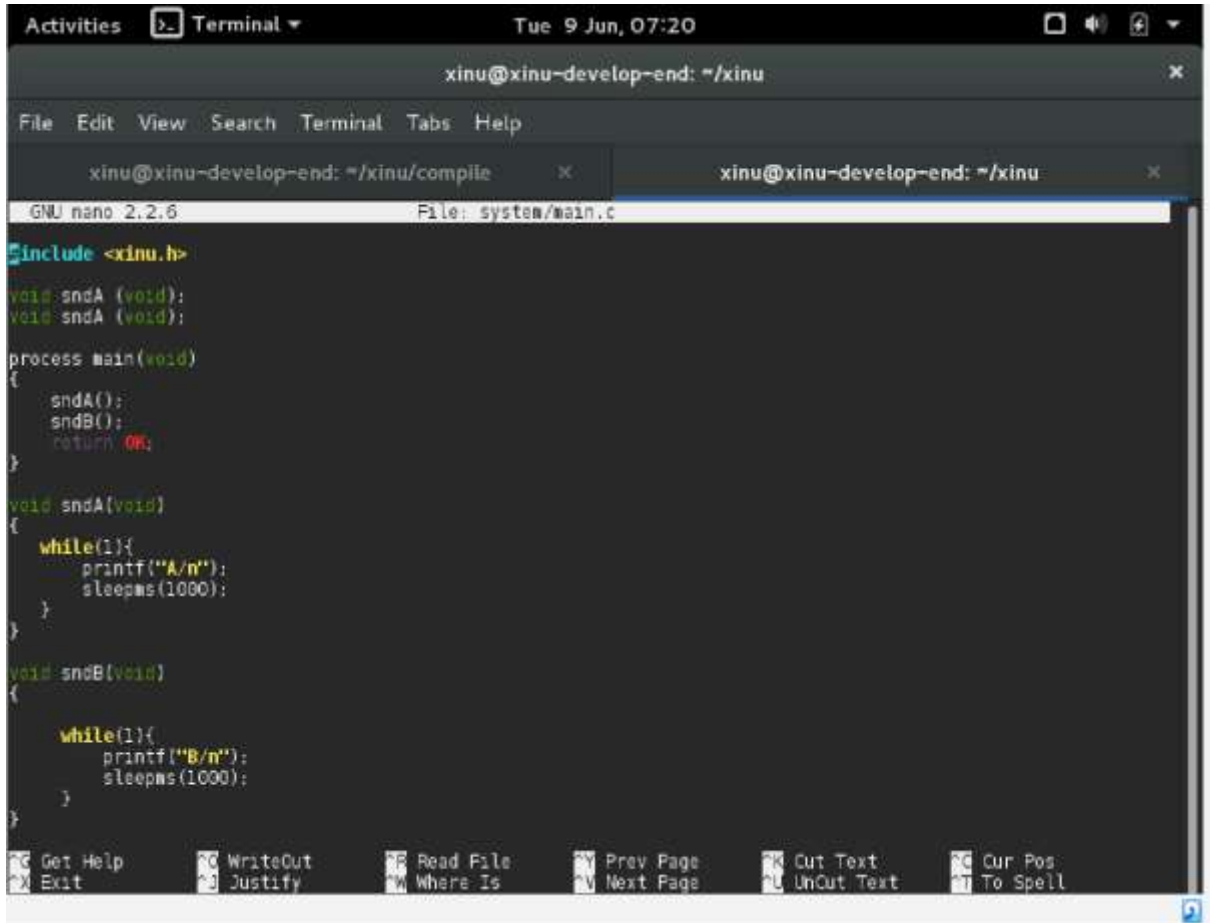
#define NPROC 150 /* number of user processes */
#define NSEM 100 /* number of semaphores */
#define IRQBASE 32 /* base ivec for IRQ0 */
#define IRQ_TIMER IRQ_HW5 /* timer IRQ is wired to hardware 5 */
#define IRQ_ATH_MISC IRQ_HW4 /* Misc. IRQ is wired to hardware 4 */
#define CLKFREQ 200000000 /* 200 MHz clock */

#ifndef ETHER0
#define ETHER0 0
#endif

C/C++/ObjC Header Tab Width: 8 Ln 84, Col 24 INS
```

2. Program hanya menampilkan karakter A secara terus menerus. Hal ini terjadi karena fungsi `sndA()` mengandung perulangan tak hingga (`while(1)`), sehingga eksekusi program tidak pernah mencapai pemanggilan fungsi `sndB()`. Oleh karena itu proses berjalan secara sekuensial dan hanya `sndA()` yang dijalankan.

Kode:



```
xinu@xinu-develop-end: ~/xinu
File Edit View Search Terminal Tabs Help
xinu@xinu-develop-end: ~/xinu/compile xinu@xinu-develop-end: ~/xinu
GNU nano 2.2.6 File: system/main.c

#include <xinu.h>

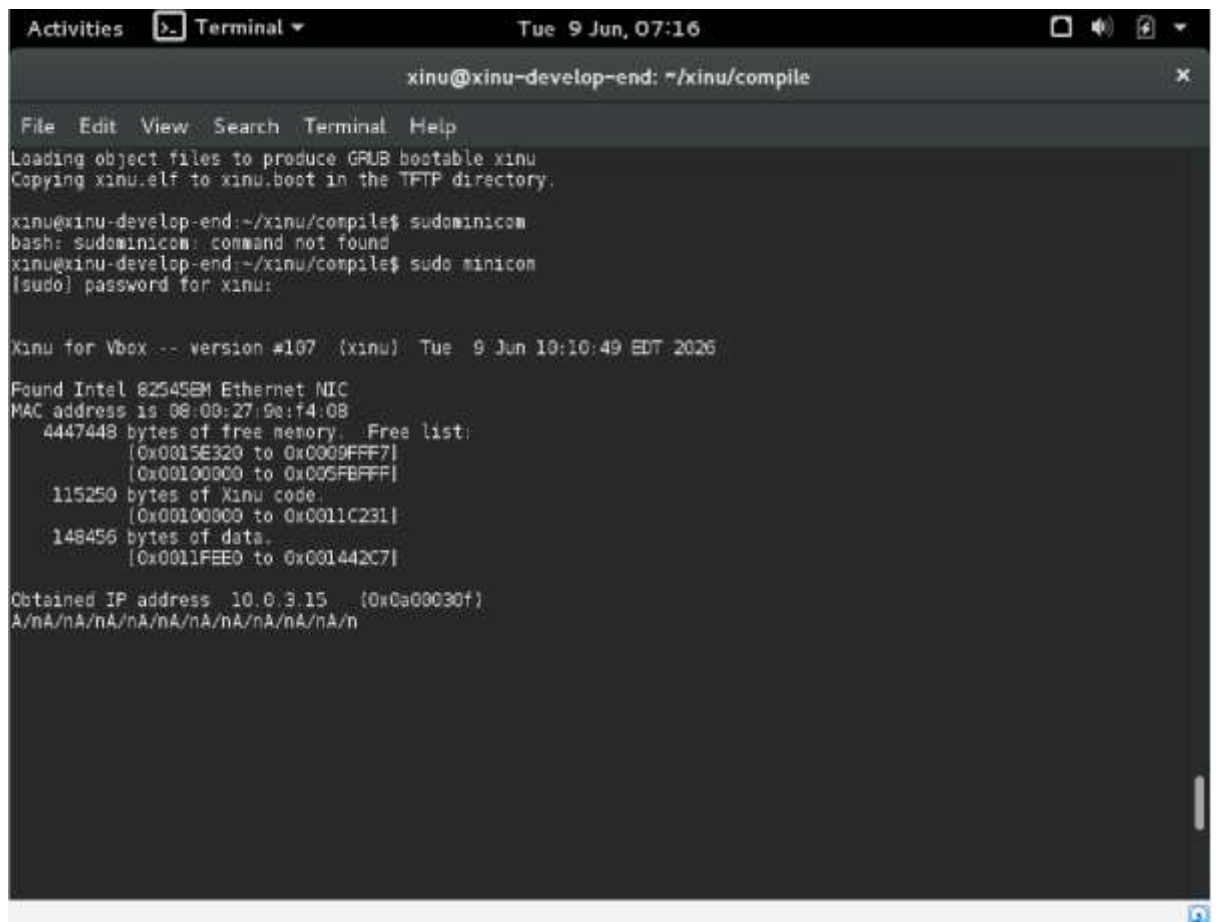
void sndA (void);
void sndA (void);

process main(void)
{
    sndA();
    sndB();
    return OK;
}

void sndA(void)
{
    while(1){
        printf("A/n");
        sleepms(1000);
    }
}

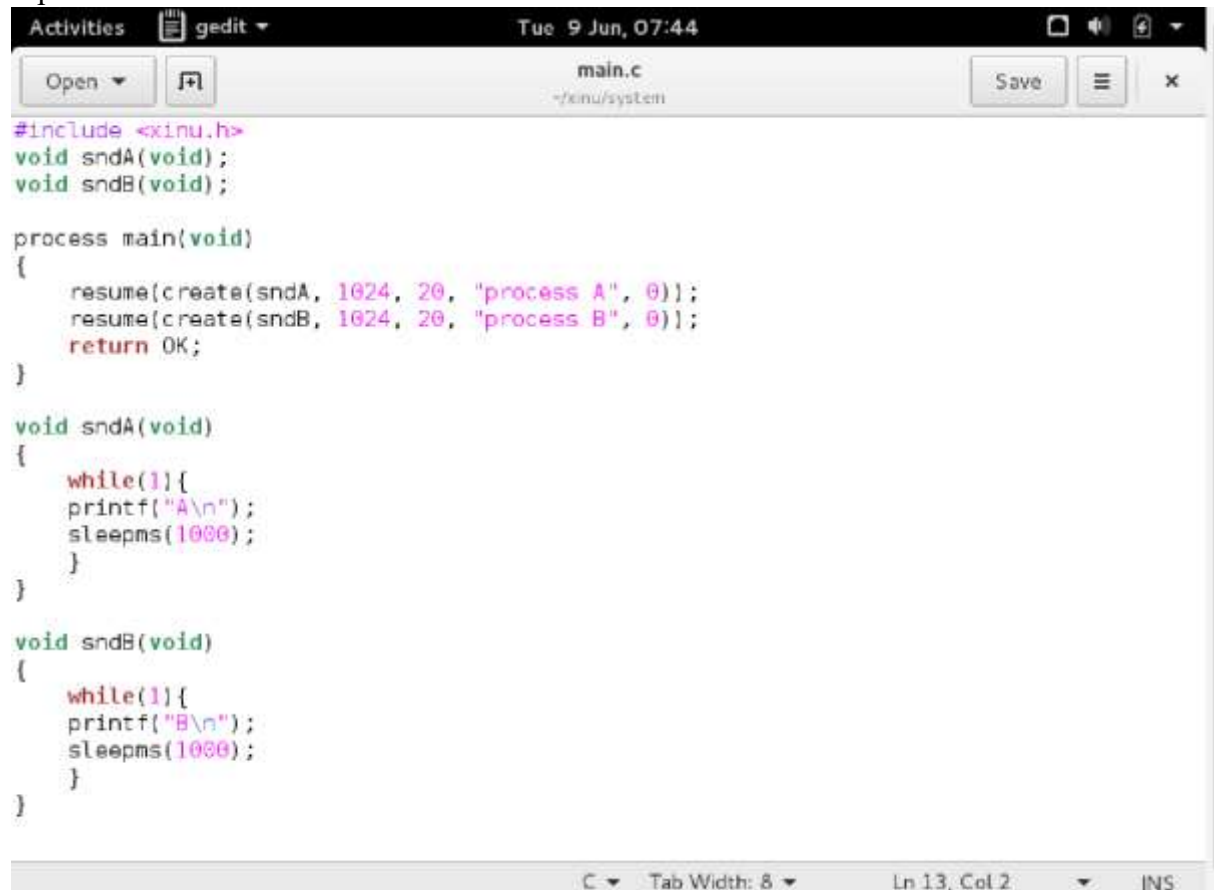
void sndB(void)
{
    while(1){
        printf("B/n");
        sleepms(1000);
    }
}
```

Hasil Output:



3. Pada program main_ex2, fungsi create() digunakan untuk membuat dua proses yaitu sndA() dan sndB(), kemudian resume() digunakan untuk menjalankan kedua proses tersebut secara konkuren. Kedua proses memiliki prioritas yang sama sehingga scheduler Xinu akan menjalankan keduanya secara bergantian. Oleh karena itu output yang dihasilkan berupa karakter A dan B yang muncul secara bergantian. Urutan kemunculan A dan B dapat berbeda-beda pada setiap eksekusi karena bergantung pada penjadwalan proses oleh sistem operasi.

Input:



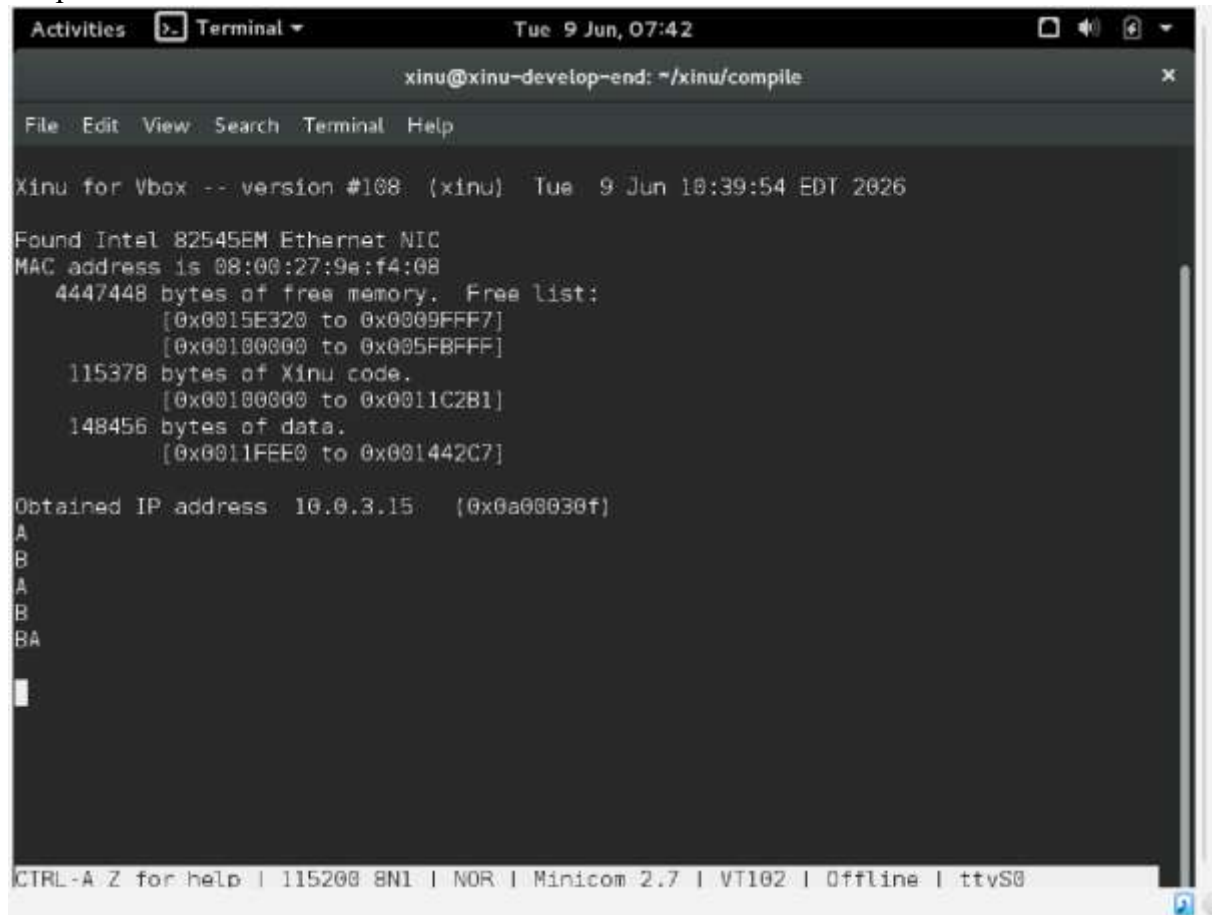
```
#include <xinu.h>
void sndA(void);
void sndB(void);

process main(void)
{
    resume(create(sndA, 1024, 20, "process A", 0));
    resume(create(sndB, 1024, 20, "process B", 0));
    return OK;
}

void sndA(void)
{
    while(1){
        printf("A\n");
        sleepms(1000);
    }
}

void sndB(void)
{
    while(1){
        printf("B\n");
        sleepms(1000);
    }
}
```


Output:

A terminal window titled 'xinu@xinu-develop-end: ~/xinu/compile' with a menu bar (File, Edit, View, Search, Terminal, Help) and a status bar (CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Offline | ttyS0). The terminal output shows the Xinu boot process: 'Xinu for Vbox -- version #108 (xinu) Tue 9 Jun 10:39:54 EDT 2026', hardware detection of an Intel Ethernet NIC, memory layout details (4447448 bytes free, 115378 bytes code, 148456 bytes data), and IP address assignment (10.0.3.15). It then displays a sequence of characters 'A', 'B', 'A', 'B', 'BA' and a cursor.

```
xinu@xinu-develop-end: ~/xinu/compile
File Edit View Search Terminal Help

Xinu for Vbox -- version #108 (xinu) Tue 9 Jun 10:39:54 EDT 2026
Found Intel 82545EM Ethernet NIC
MAC address is 08:00:27:9e:f4:08
  4447448 bytes of free memory. Free list:
    [0x0015E320 to 0x0009FFF7]
    [0x00100000 to 0x005FBFFF]
  115378 bytes of Xinu code.
    [0x00100000 to 0x0011C2B1]
  148456 bytes of data.
    [0x0011FEE0 to 0x001442C7]

Obtained IP address 10.0.3.15 {0x0a00030f}
A
B
A
B
BA
█

CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Offline | ttyS0
```

4. Pada program main_ex3, digunakan satu fungsi yang sama yaitu sndChar(char c). Dua proses dibuat menggunakan fungsi create(), namun masing-masing diberikan parameter yang berbeda yaitu 'A' dan 'B'. Kedua proses dijalankan secara konkuren menggunakan resume(). Karena kedua proses berjalan secara bersamaan dan memiliki prioritas yang sama, scheduler Xinu akan mengatur pergantian eksekusi proses sehingga karakter A dan B muncul secara bergantian pada layar.

Input:



```
#include <xinu.h>
void sndChar(char);

process main(void)
{
    resume(create(sndChar, 1024, 20, "Send A", 1, 'A'));
    resume(create(sndChar, 1024, 20, "Send B", 1, 'B'));
    return OK;
}

void sndChar(char c)
{
    while(1){
        printf("%c \n", c);
        sleepms(1000);
    }
}
```

Output:

```
Activities Terminal Tue 9 Jun, 07:57
xinu@xinu-develop-end: ~/xinu/compile

File Edit View Search Terminal Help
xinu@xinu-develop-end:~$ cd ~/xinu/compile
xinu@xinu-develop-end:~/xinu/compile$ sudo minicom
[sudo] password for xinu:

Xinu for Vbox -- version #109 {xinu} Tue 9 Jun 10:54:04 EDT 2026

Found Intel 82545EM Ethernet NIC
MAC address is 08:00:27:9a:f4:08
  4447448 bytes of free memory. Free list:
    [0x0015E320 to 0x0009FFF7]
    [0x00100000 to 0x005FBFFF]
  115346 bytes of Xinu code.
    [0x00100000 to 0x0011C291]
  148456 bytes of data.
    [0x0011FEE0 to 0x001442C7]

Obtained IP address 10.0.3.15 {0xa00030f}
A
B
A
B
AB
B
A
B
A
```

5. Hasil program tidak selalu sesuai dengan intuisi karena proses produser dan konsumen berjalan secara konkuren dan mengakses variabel global `n` secara bersamaan. Konsumer dapat membaca nilai `n` ketika produser belum selesai melakukan increment sebanyak 1000 kali. Akibatnya nilai yang ditampilkan dapat berupa 0, nilai antara 1–999, maupun 1000. Hal ini terjadi karena tidak ada mekanisme sinkronisasi seperti semaphore atau mutex yang mengatur akses terhadap variabel bersama. Kondisi tersebut disebut **race condition**, yaitu ketika dua atau lebih proses mengakses data yang sama secara bersamaan sehingga hasil yang diperoleh bergantung pada urutan eksekusi proses oleh scheduler sistem operasi.

Input:



```
Activities Terminal Tue 9 Jun, 14:28
xinu@xinu-develop-end: ~/xinu

File Edit View Search Terminal Help

GNU nano 2.2.6 File: system/main.c
#include <xinu.h>

int32 n = 0;

void producer(void)
{
    int32 i;
    for(i=1; i<=1000; i++)
    {
        n++;
    }
}

void consumer(void)
{
    int32 i;
    for(i=1; i<=1000; i++)
    {
        printf("Nilai dari n adalah %d\n", n);
    }
}

process main(void)
{
    resume(create(producer, 1024, 20, "Producer", 0));
    resume(create(consumer, 1024, 20, "Consumer", 0));
    return 0;
}
```

Output:

